File Systems
— Chapter 10

叶冠华

g.ye@bupt.edu.cn

School of Computer Science (National Pilot Software Engineering School)

北京郵電大學

Review

- 虚拟内存
- Demand Paging: page fault
- Page Replacement Algorithms: FIFO, OPT, LRU, Second Chance
- Allocation of Frames
- Copy-on-write, Memory-mapped-file
- Allocation kernel memory: Buddy system

Course Architecture

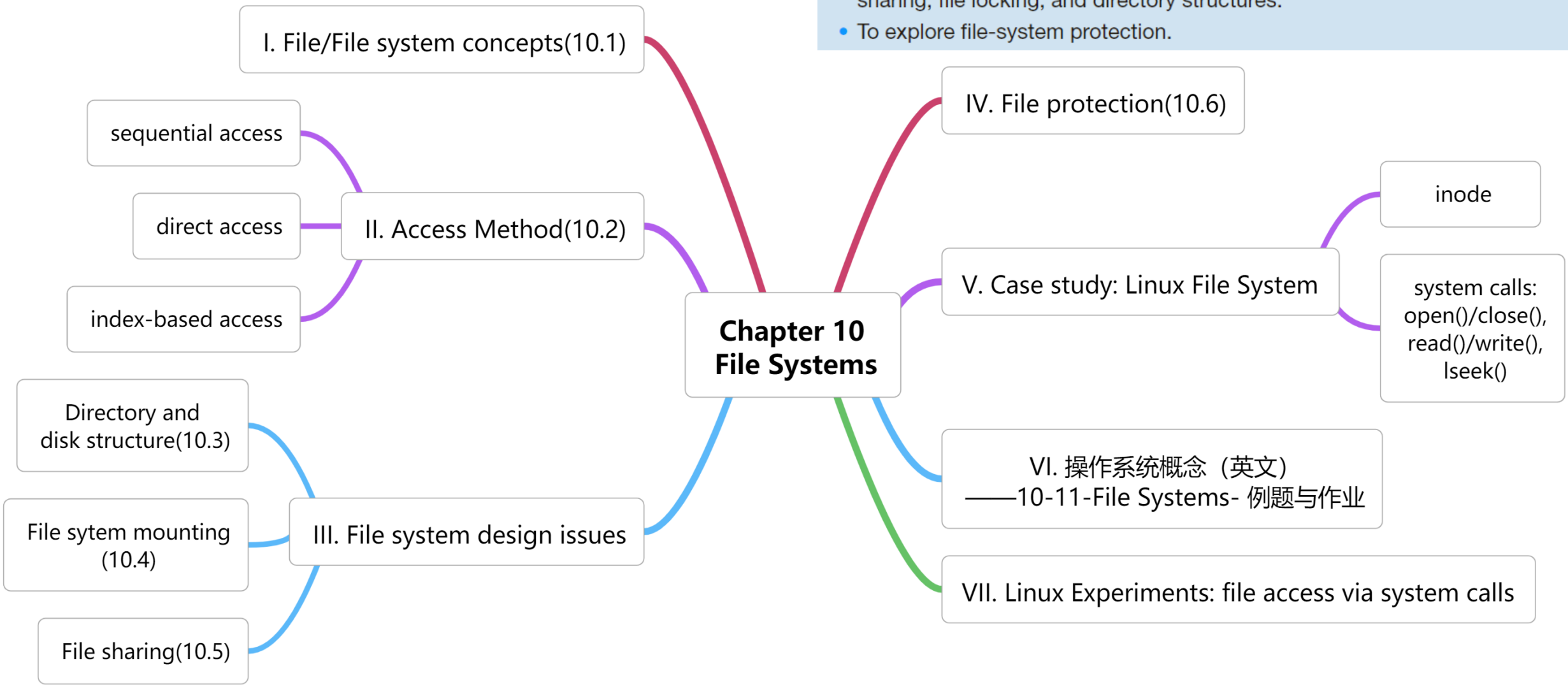
- 1 Introduction
- 2 System Structures
- 3 Process Concepts — Virtualization
- 4 Multi-threading — Concurrency
- 5 Process Scheduling — Virtualization
- 6 Process Synchronization — Concurrency
- 7 Deadlocks — Concurrency
- 8 Memory Management — Virtualization
- 9 Virtual Memory — Virtualization

---------- We are here ----------

- 10 File Systems — Persistence
- 11 Implementing File Systems — Persistence
- 12 Mass Storage Structure — Persistence
- 13 IO Systems — Persistence

CHAPTER OBJECTIVES

- To explain the function of file systems.
- To describe the interfaces to file systems. system call
- To discuss file-system design tradeoffs, including access methods, file sharing, file locking, and directory structures.
- To explore file-system protection.



Chapter 10-11
File Systems例题与作业

- Example 1 文件访问逻辑字节/记录 → 物理块
- 文件分配方式/物理结构 → 文件数据块访问方式
  - Example 2 Contiguous/linked file, Read
  - Example 3 Contiguous/linked/indexed file, Insert/Delete
- Example 4-1/4-2 位图/位向量(Bitmap/Bitvector)
- Linked file
  - Example 5 根据地址指针长度计算maximum file size
  - Example 6 计算maximumfile size；逻辑字节对应的I/O次数（数据块访问方式）
- Indexed file
  - Example 7 根据索引结构计算maximumfile size
  - Example 8 Indexed file: 根据索引结构计算访问block数目
  - Example 9 Indexed file: 根据索引结构计算file size/平均访问block数目
  - Example 10 Indexed文件: 根据目录结构计算所访问的目录及文件block数目
- Example 11 文件命名与目录结构
- Example 12 408选择题

Terminology

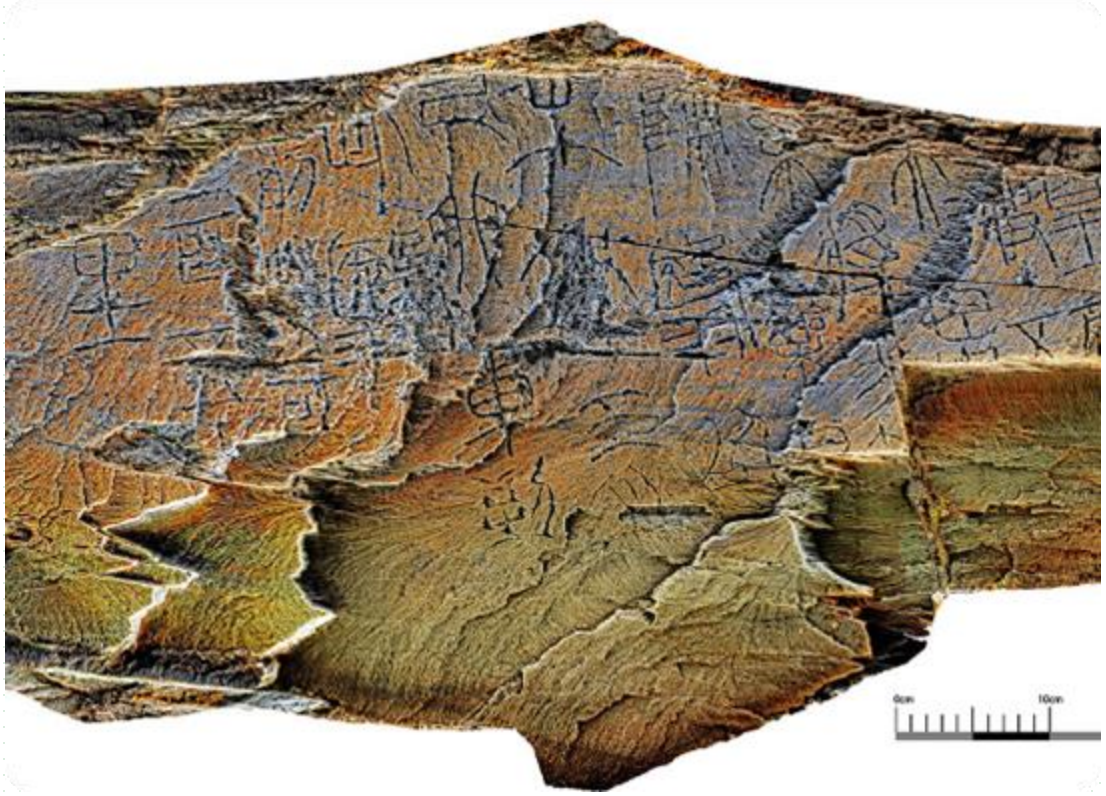
sector, block, Cluster, extent	扇区, 磁盘块, 簇, 区段,扩展块
FCB, File Control Block	文件控制块
inode	索引节点
delete, truncate	删除, 截断/清空
sequential access, direct/relative/random access, index-based access	顺序访问, 直接/相对/随机访问, 索引访问
disk partitions, volume, volume table of contents	磁盘分区, 卷, 卷目录表
mount	挂载
hard link, soft link	硬链接, 软链接

Beyond Volatile Memory

To achieve "Persistence", we obviously need a storage medium different from volatile memory (RAM).

Humans have sought to preserve information since ancient times. Consider the **Kunlun Stone Carvings**, believed to be left by Qin Dynasty officials seeking

Information carved into stone has survived for thousands of years. This is the most primitive, yet effective form of persistence.



Stone Inscriptions (Qin Dynasty Era)

Immutable Persistence

Optical Disc "Burning"

Like carving stone, lasers create physical pits on the disc surface. This is a **Write-Once** medium: you can append, but you cannot delete.

Log Files (Append-Only)

Modern Computer Science inherits this trait in **Logs**. To ensure data integrity and traceability, many critical data structures are designed as "Append-Only".

Editable Persistence

The Magnetic Drawing Board

The pen tip is a magnet: it pulls black magnetic powder up (Writing a **1**).

The slider is a magnet: it pulls the powder back down (Resetting to **0**).

Key Concept: **Reversible State Change**.

Editable Persistence

Managing 0s and 1s

Mechanical Hard Drives (HDD) work on principles strikingly similar to that toy. Instead of a plastic pen, we use a microscopic **Magnetic Head**.

Tiny regions (Bits) on the platter can have their magnetic polarity flipped to represent 0 or 1. This allows us to persist data while retaining the ability to modify and overwrite it.

This is the physical foundation of modern file systems: **Rewritable Persistent Media**.

Key Question

Why the difference?

Since both Memory and Disk manage combinations of 0s and 1s...

"We've already learned address translation and page tables. Why can't we just manage the disk using physical addresses directly?"

Software Perspective

The fundamental difference lies in the interface. Humans cannot remember chaotic binary addresses (e.g., 0x8000FF). We need **Names**.

Filenames

One of the greatest OS abstractions. To support it, we need rules (length limits) and conflict resolution.

Directories

When names run out, we use folders. Logically similar to multi-level page tables—a hierarchical mapping.

Metadata

A file is more than data. It has owners, timestamps, and permissions. We need structures to store this context.

Hardware Perspective

The Granularity Gap

Hardware characteristics dictate why we cannot simply copy memory management techniques.

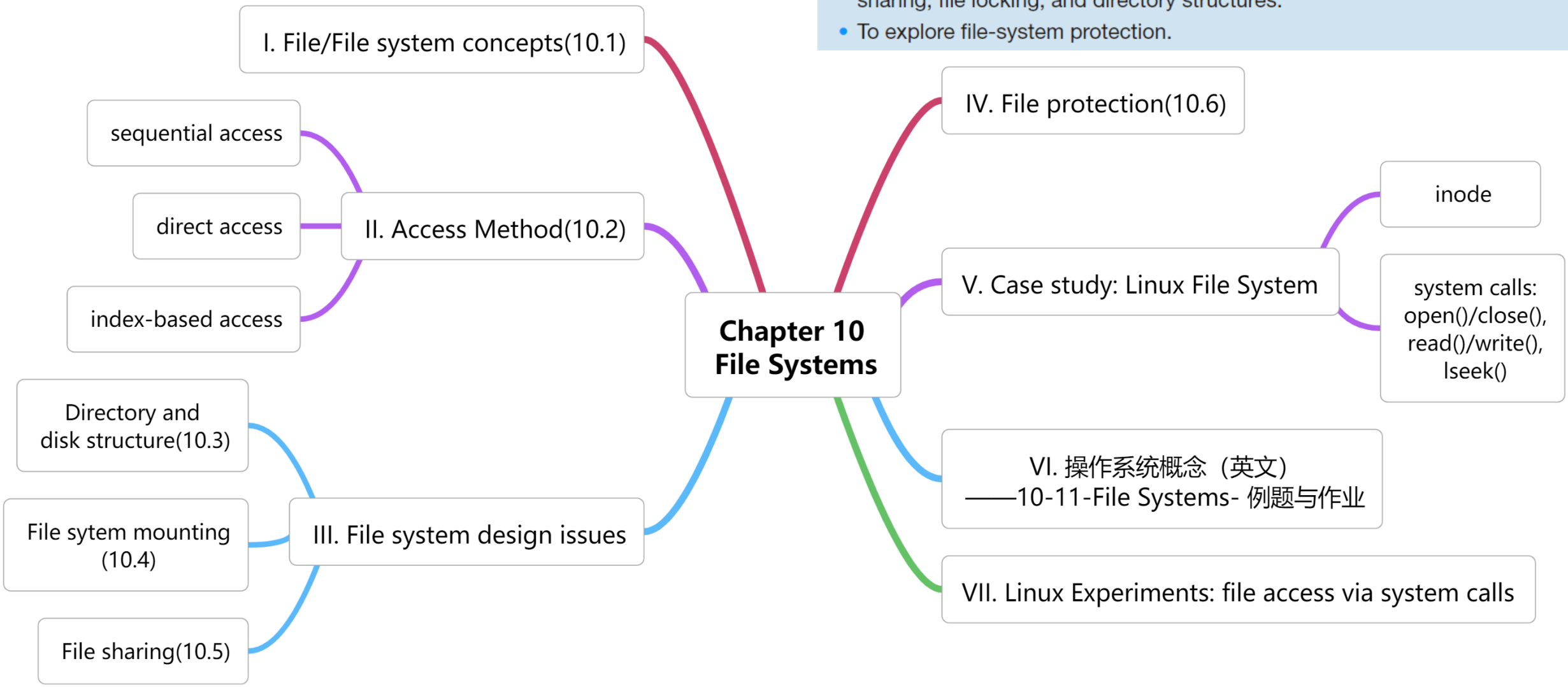
Memory (RAM): Is **Byte-addressable**. The CPU can fetch any single byte directly.

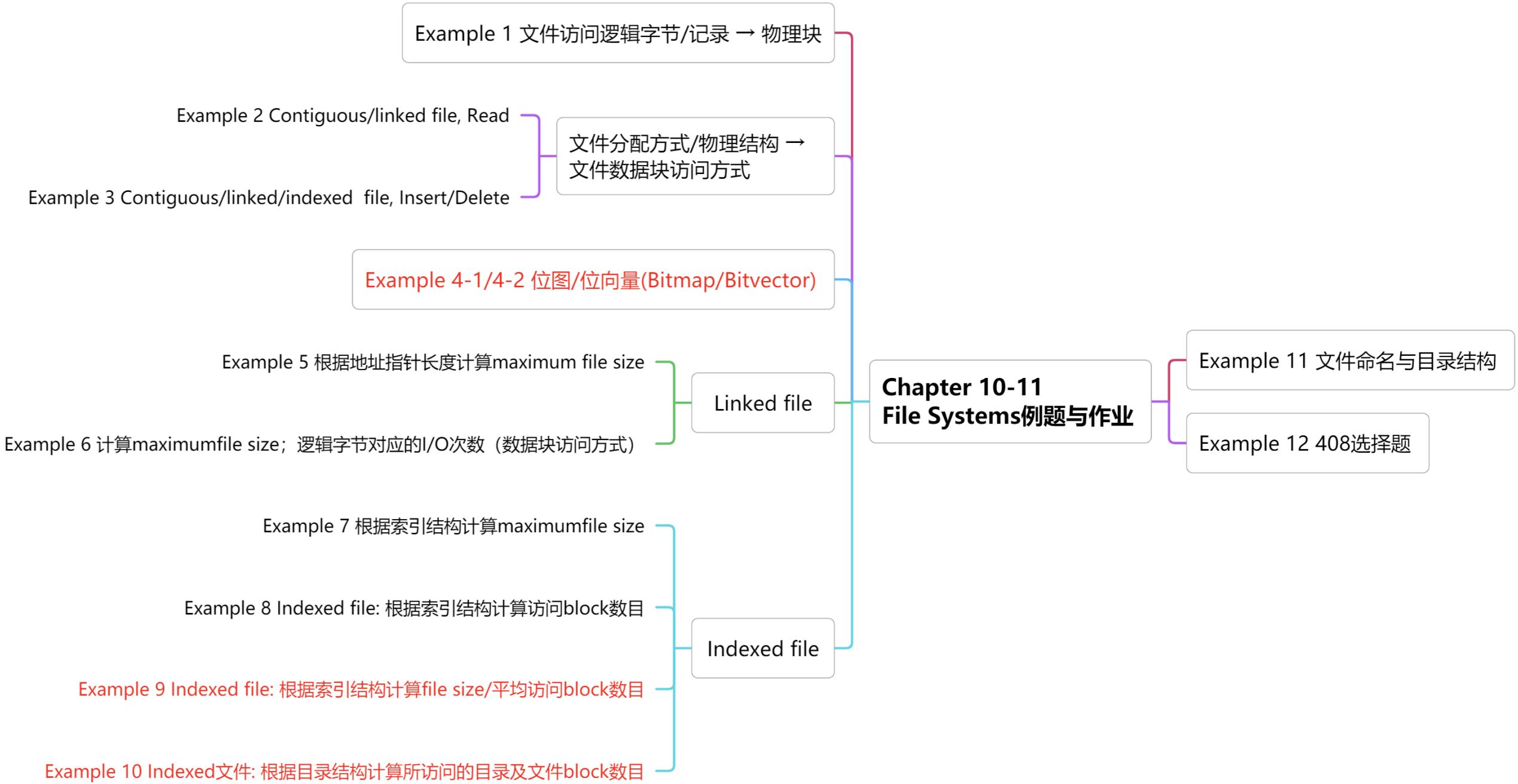
Disk (Storage): Is **Block-addressable**. You cannot read just the 3rd byte; you must fetch the entire 4KB block into memory first.

This latency and granularity gap necessitates complex **Buffer Caches** in the OS.

CHAPTER OBJECTIVES

- To explain the function of file systems.
- To describe the interfaces to file systems. system call
- To discuss file-system design tradeoffs, including access methods, file sharing, file locking, and directory structures.
- To explore file-system protection.




Example 1 文件访问逻辑字节/记录 → 物理块
Example 2 Contiguous/linked file, Read
文件分配方式/物理结构 → 文件数据块访问方式
Example 3 Contiguous/linked/indexed file, Insert/Delete
Example 4-1/4-2 位图/位向量(Bitmap/Bitvector)
Example 5 根据地址指针长度计算maximum file size
Linked file
Example 6 计算maximumfile size；逻辑字节对应的I/O次数（数据块访问方式）
Example 7 根据索引结构计算maximumfile size
Example 8 Indexed file: 根据索引结构计算访问block数目
Indexed file
Example 9 Indexed file: 根据索引结构计算file size/平均访问block数目
Example 10 Indexed文件: 根据目录结构计算所访问的目录及文件block数目
Chapter 10-11
File Systems例题与作业
Example 11 文件命名与目录结构
Example 12 408选择题

10.1 File

- **File** Definition(P456 in Edt.9)
  - a named collection of related information that is recorded on secondary storage, i.e. disk, SSD/Flash memory tape etc.
  - commonly, representing programs and data
  - e.g. C++ source program file, executable file, text file
- **File Definition** in Microsoft Computer Dictionary
  - a file is the “glue” that binds a conglomeration of instructions, numbers, words, or images into a coherent unit that a user can retrieve, change, delete, save, or sent to an output devices

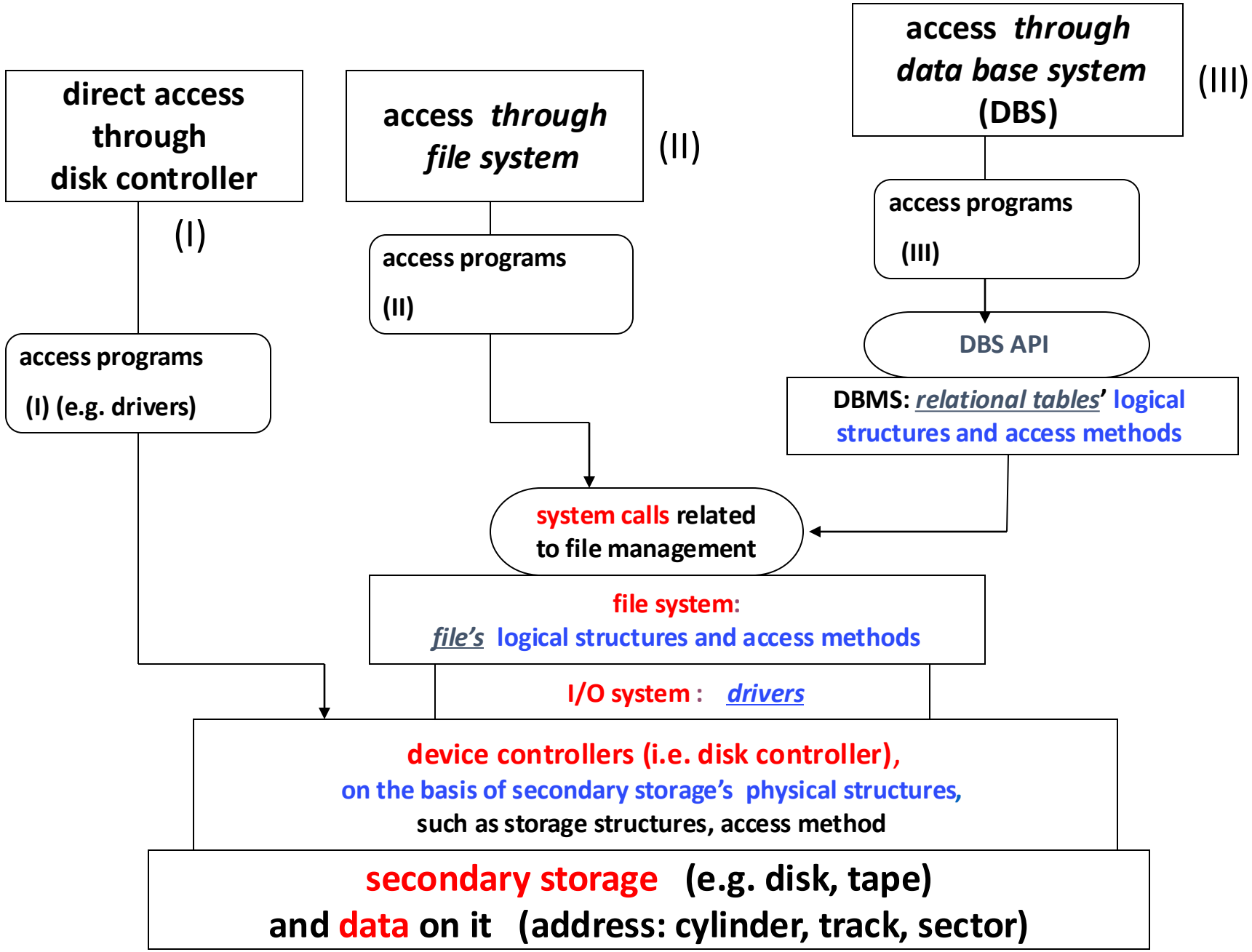
File System

- **Definition of File System (**P453 in Edt.9**)**
  - a software component in operating systems, providing the mechanism for on-line storage of and access to both data and programs residing on disk/secondary storage
  - in general, also containing
    - a collection of files
    - a directory structure
- **File System *Definition*** in *Microsoft Computer Dictionary*
  - in a operating system, the overall structure in which files are named, stored, and organized
  - file system consists of files, directories, and the information needed to locate and access these items
  - the term *can also refer to the portion of an operating system* that translates requests for file operations from an application program into low-level , sector-oriented tasks that can be understood by the drivers controlling the disk drives

Why file systems needed

- file vs. file system, similar to DB vs. DBMS
- Access Program (I)
  - need to know physical organization access of data on secondary storage
- Access program (II) based on file systems, independent of hardware (i.e. disk, device controller), but related to particular operating systems
  - if OS changes, program (II) also changes
  - file systems provide unified access interfaces between users and data on secondary memory, on the basis of files' logical structures and access methods defined by OS itself
  - the access interfaces may vary with different OS. But for one OS, its access interfaces are identical with respect to different secondary memories and memory controllers

    /*利用OS 的file system, I/O system, 隐藏了不同外存的物理存储和访问差异*/


direct access through disk controller
(I)
access through file system
(II)
access through data base system (DBS)
(III)
access programs (I) (e.g. drivers)
access programs (II)
access programs (III)
DBS API
DBMS: relational tables' logical structures and access methods
system calls related to file management
file system: file's logical structures and access methods
I/O system : drivers
device controllers (i.e. disk controller), on the basis of secondary storage's physical structures, such as storage structures, access method
secondary storage (e.g. disk, tape) and data on it (address: cylinder, track, sector)

File Concept

- From viewpoints of users, a file consists of a contiguous logical address space, containing
  - a set of structured records reside, #record as logical address

0	1	2	…	k	…		n

//称为有"结构"的记录文件

    - e.g. database file storing relational table, with tuples/rows in table as records in the file
    - e.g. slides in PPT document
  - bytes/words
    - e.g. xxx.txt file, executable 0-1 code programs; or

//后两类称为无结构的流式文件

  - storage unit of fixed-size, or logical block
    - such as ***extent*(区段，扩展块)** of 1024-byte in size in Unix/Linux, ext2 and ext4 file system

File Attributes

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Many variations, including extended file attributes such as file checksum

File info on Mac OS

(General) File Control Block

- 广义FCB:
  - Linux -> Inode (we will learn this in the next lesson)
  - Windows(NTFS) -> MFT Entry
  - DOS -> FCB (狭义FCB)

(Special) File Control Block

- 狭义FCB: The FCB originates from CP/M and is also present in most variants of DOS. A full FCB is 36 bytes long. This fixed size, which could not be increased without breaking application compatibility, led to the FCB's eventual demise as the standard method of accessing files.
- Information about files are kept in the directory entries (目录项) stored in directory structure, which is maintained on the disk

Offset	Byte size	Contents
0x00	1	Drive number — 0 for default, 1 for A:, 2 for B:, ...
0x01	8	File name and extension — together these form a 8.3 file name.
0x09	3	
0x0C	20	Implementation dependent — should be initialised to zero before the FCB is opened.
0x20	1	Record number in the current section of the file — used when performing sequential access.
0x21	3	Record number to use when performing random access.

Offset	Byte size	Contents
0x0E	2	File's record length in bytes.
0x10	4	Total file size in bytes.
0x14	2	Date of last modification to file contents.
0x16	2	Time of last modification.

File Operations

- **Create**
  - allocate secondary storage space to the created file F_i
  - create and insert new FCB entry for F_i corresponding to the file into directory
- **Current-file-position-pointer** for file operations, read/write pointer
  - pointer to the file data to be read or wrote
  - e.g. cfo (current file offset) in Linux, refer to Fig. 10.4
- **Write(**file, information to be wrote**) –** at **write pointer** location
- **Read(**file, buffer-address**)–** at **read pointer** location
- **Reposition within file –** also known as file **seek,重定位**
  - setup or move current-file-position-pointer, e.g. cfo, for file operations

File Operations

- Delete
  - erase space allocated to the file
  - erase the directory entry of the file
- Truncate(清空、截断)
  - erase the contents of a file but keeps its attributes

- The six operations mentioned above comprise the minimal set of required file operations
- Other operations
  - appending, renaming, copy

Open/Close Files

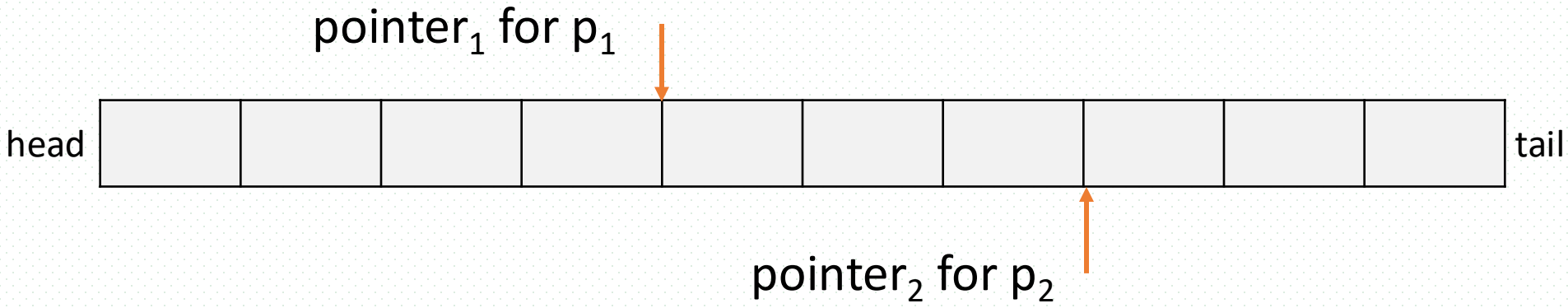
- File operations mentioned involve searching the directory for the entry associated with the named file. To avoid searching, an open-file table(打开文件表，≈ working set in VM) is introduced in operating systems, which contains information about all opened files
  - open-file table: a data structure in main memory, to record the directory entries of files
- Two operations associated with open-file table are defined
- **Open** (F_i)
  - search the directory structure on the disk for entry F_i, and copy the content of the entry into the open-file table in main memory
  - In Linux/Unix, 将文件的读写指针cfo(current file offset)初始化为0，指向文件头部
- **Close** (F_i)
  - remove the entry F_i in the open-file table
  - or move the content of entry $\boldsymbol{F_i}$ in memory to directory structure on disk

Open/Close Files

- In the open-file table, information associated with the opened file
  - the disk location of the file
  - the access rights
  - the file open count
    - the number of processes which open the file
    - e.g. two processes p_1 and p_2 open a file, then count=2 for this file
  - the file pointer, e.g. cfo in Linux/Unix files

  – indicating the last ***read-write*** location in the file

  – unique to each process operating on the opened file



An opened file on disk

Open File Locking

- Some OS provide file locking to control concurrently accessing of files by processes
  - e.g. multiple processes access system log file

    several transactions write log files in database system (DBS)
  - Similar to locking in the **reader-writer problem**
  - **Shared lock** similar to reader lock – several processes can acquire concurrently
  - **Exclusive lock (独占、互斥锁)** similar to writer lock
- Mediates access to a file
- Mandatory(强制) or advisory(建议)
  - **Mandatory** – access is denied depending on locks held and requested, e.g. in Windows

    //由OS提供完全的locking
  - **Advisory** – processes can find status of locks and decide what to do, in Unix

    //programs与OS一起，实现对文件的locking

File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	read to run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rrf, doc	various word-processor formats
library	lib, a, so, dll, mpeg, mov, rm	libraries of routines for programmers
print or view	arc, zip, tar	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm	binary file containing audio or A/V information

Fig.10.3 Common file type

- Note
  - operating systems and system programs cooperatively support and implement some complex types of files, especially **record-base files**
  - Examples
    - database file supported by DBMS and OS
    - PPT document implemented by Office
    - multimedia files, such as mp4

Locking on data objects in DBS [略]

- DBMS uses locking schemes to control transactions to concurrently access data items/objects in database
- Multiple-granularity data objects/items
  - database, table, tuple, attributes; file, page, record, …
- Locking in DBS
  - lock types
    - shared lock, exclusive lock, intension lock
  - lock granularity
    - locking on data objects with different sizes
  - multiple-version scheme
    - for data item Q, there are several copies of Q, e.g. $\{Q_0, Q_1, \ldots, Q_i, \ldots, Q_j, \ldots\}$, write(Q) issues a new version of Q, e.g. Q_j, and read(Q) operation can read Q on other version, e.g. Q_i
- For more details, refer to ***Database System Concepts***

File (Logical) Structure

Types of file structures

- 1. none-structure/stream-structure
  - file is viewed as a sequence of words, bytes, e.g.
    - executable file viewed as 0/1 strings
    - text file viewed as 8-bit byte strings
- 2. Simple record structure
  - file consists of records with specific structure, e.g.
    - PPT document, consisting of slides
    - database file, storing tuples(元组) as record in file
  - the record in file may be
    - fixed length, or variable length
- 3. Complex structures
  - formatted document, e.g. XML
  - relocatable load file, e.g. ELF file in Linux and PE file in Windows.

File (Logical) Structure

- Can simulate last two, i.e. simple record structure and complex structure, with first method by inserting appropriate control characters
- Who decides
  - Operating system
  - Program

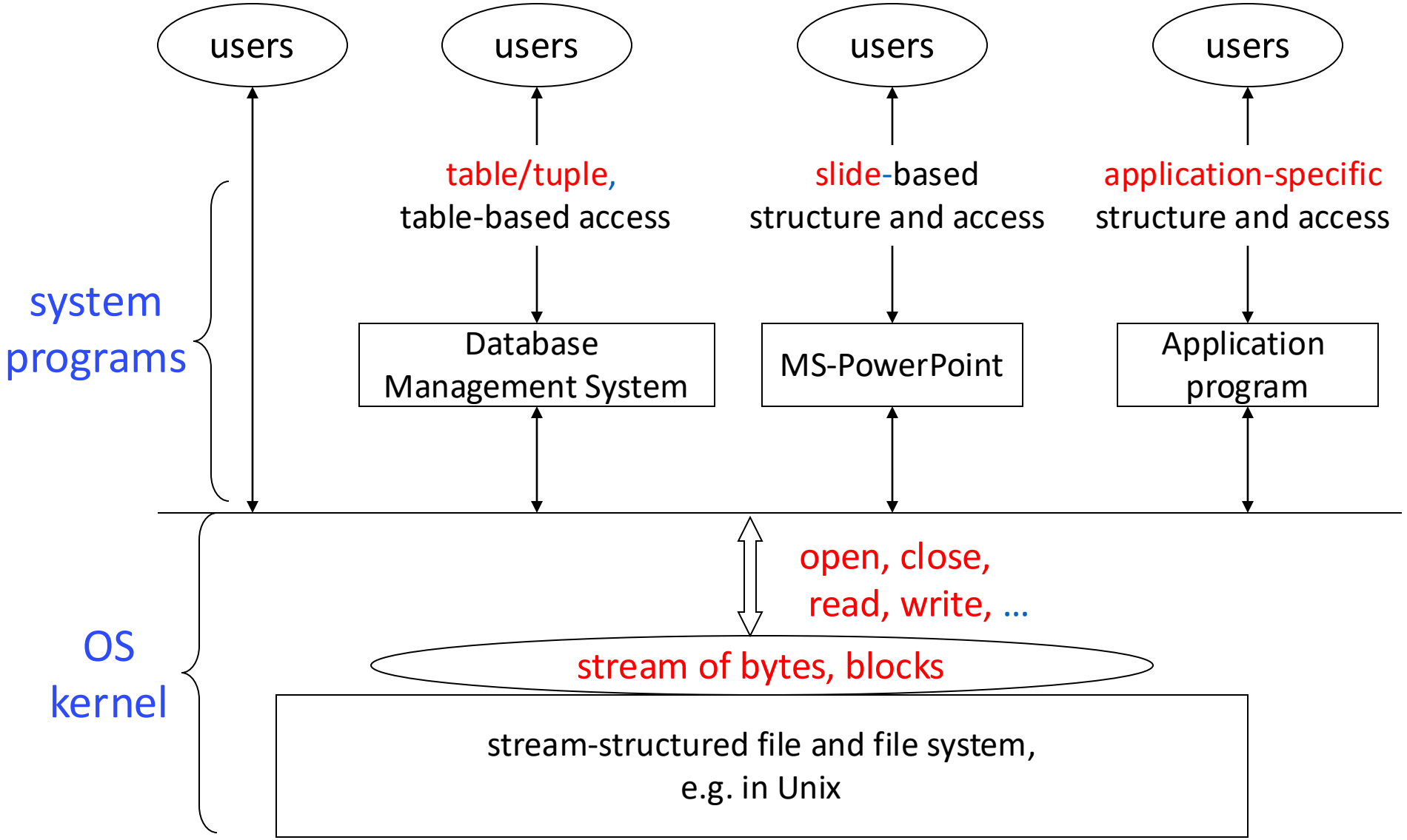


Fig.A.2 High-level interpretation of stream-structured files

instructor: ID(varch) name(varch) dept(varch) **salary(int)**

record 0	10101	Srinivasan	Comp. Sci.	65000

A variable-length record in database file [略]

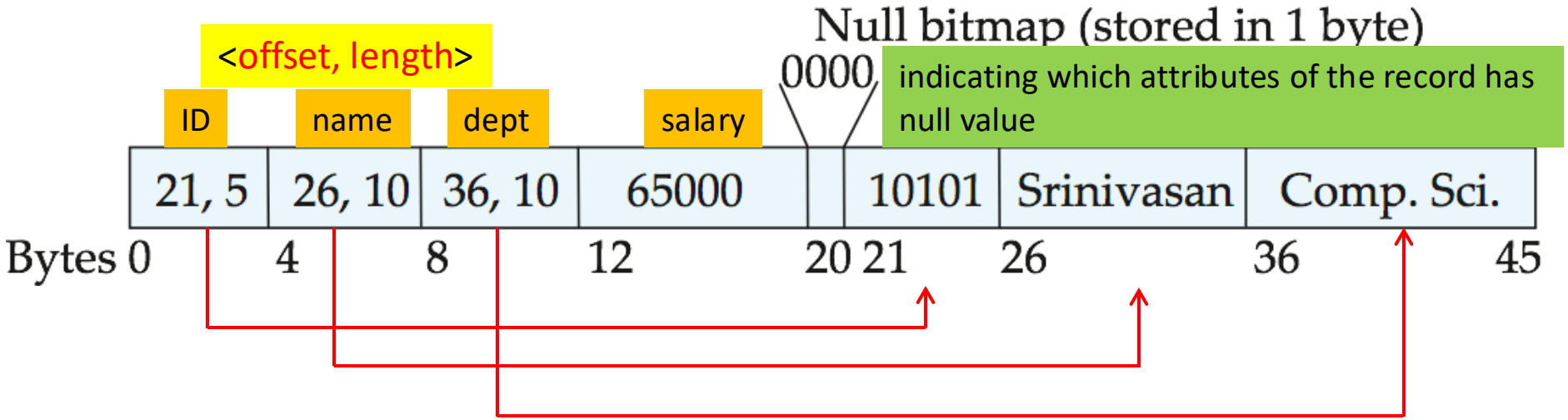


Fig. record 0 is stored in contiguous 46 bytes (0-45)

- Variable length attributes represented by fixed size (offset, length), with actual data stored after all fixed length attributes
- Null values represented by null-value bitmap
- The **slotted page** (e.g. SQL Server2008/201x) structure is often used to organize variable-length records

Four variable-length records stored in a block (slotted page) in database file

Variable-Length Records: Slotted Page Structure

- The storage space is divided into fixed-sized *slotted pages*, e.g. 4KB or 8KB, and records are allocated to slotted pages, e.g. MS SQL Server

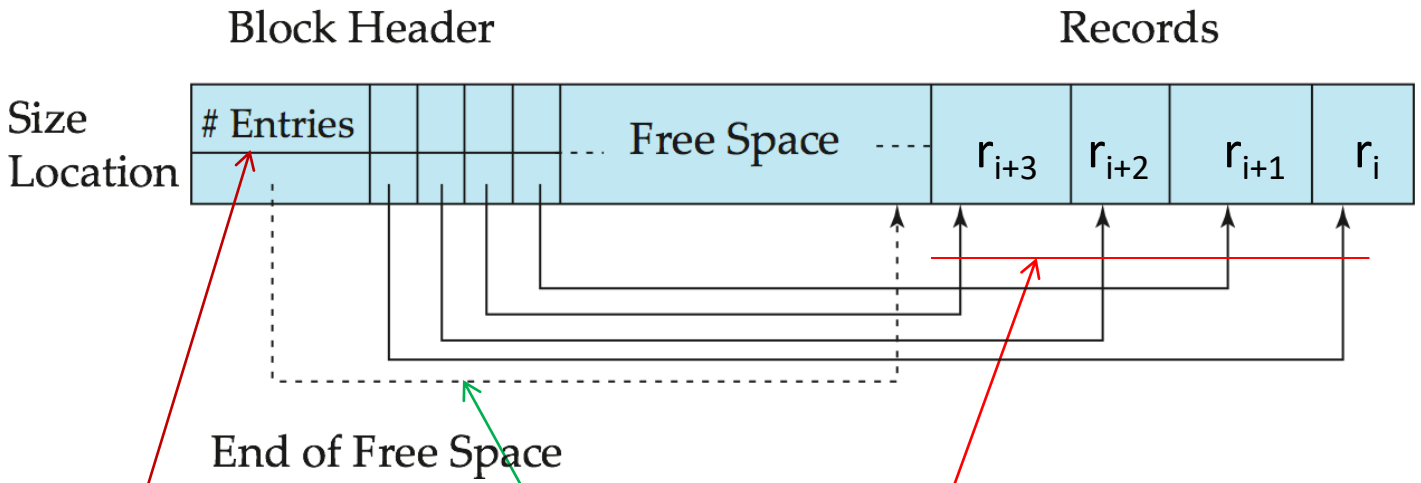


Fig. $record_i$ to $record_{i+3}$ in a page

- **Slotted page** header contains
  - number of record entries, e.g. 4
  - end of free space in the block
  - location and size of each record

File Structure

- 为了简化系统，大多数现代操作系统（内核）只/主要提供简单的流式文件结构，记录式文件等更为复杂的数据组织结构由操作系统之上的系统软件（DBMS, MS-Word, MS-Excel）或应用程序通过对流式数据的进一步结构化解释来得到
  - OS支持过多的文件系统类型会复杂化OS的设计

- File's logical structure vs physical structure
  - logical records in files are stored in physical disk ***blocks***

    records ⟺ block

  - a file can also be viewed as a sequence of physical blocks
    - physical structures of the file
- Logical structure determines the user-oriented file system ***interfaces***
  - OS按照文件逻辑结构，e.g. records, bytes, extent, 对用户提供文件访问系统调用

10.2 Access Methods

- Three types of user-oriented access methods
  - Sequential Access
  - Direct Access (relative/random access)
  - Index-based access

- Provided by system calls
  - e.g. in Linux: open, close, write, read, lseek

Sequential Access

- Sequential access is based on the ***tape*** model of files, commonly-used by editors and compilers
- Data organization
  - information in the file is arranged in order, one byte/word/record after another, according to **current-position-pointer**， e.g. ***cfo*** in Linux in Fig. 10.4
- File operations
  - *read next*
    - read the next portion of the file, and then advances a file pointer

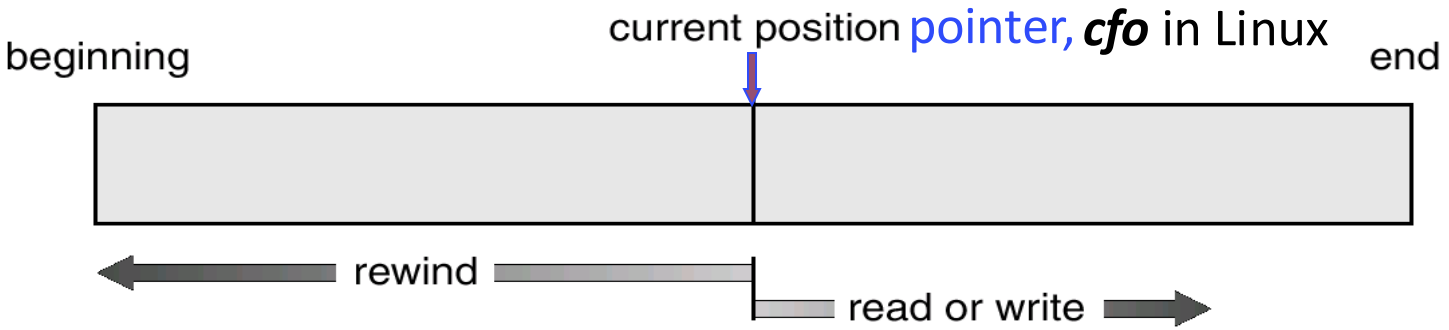
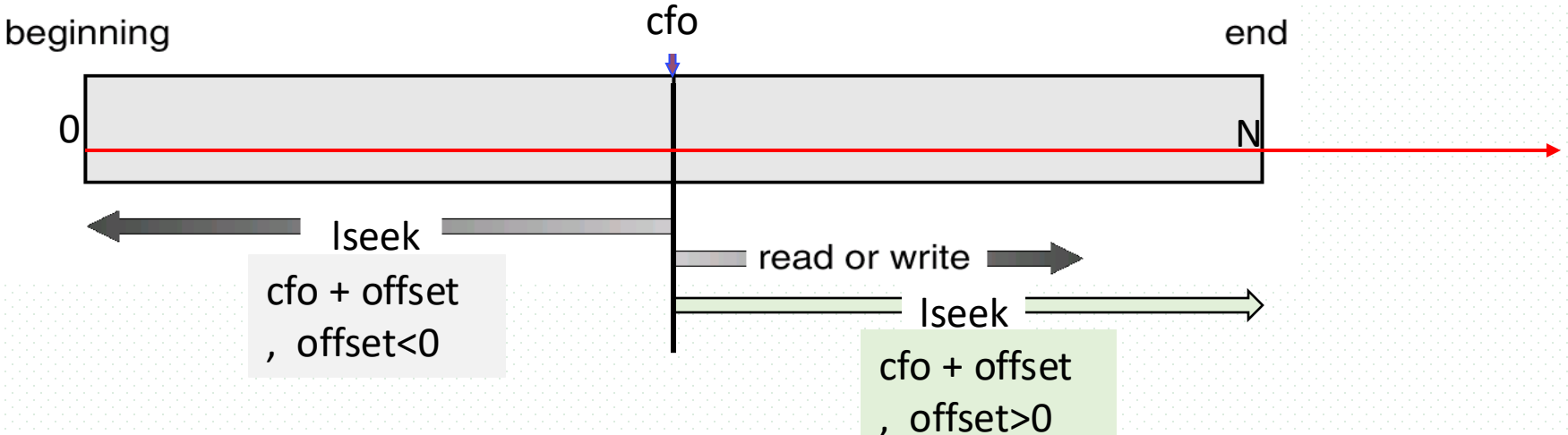


Fig. 10.4 Sequential-access file

当前文件偏移量cfo(current file offset) in Linux

- cfo
  - 文件读写指针位置，即文件开始位置到文件当前访问位置的字节数，一般是一个非负整数
  - 文件的read()、write()始于cfo，即根据cfo指向的文件位置，进行读写操作，并使cfo值增大，cfo的增量为读写的字节数
- open()操作打开文件时，文件的cfo初始化为0，指向文件开始位置，除非参数flags=O_APPEND
- 使用系统调用lseek(..., offset, ...)可以改变文件cfo, 即改变文件读写指针位置



Sequential Access

- write next
  - appends to the end of the file (在文件尾部写入数据), and advances the new end of the file
- Reset
  - reset **current-position-pointer** to the beginning, skip forward or backward *n* records

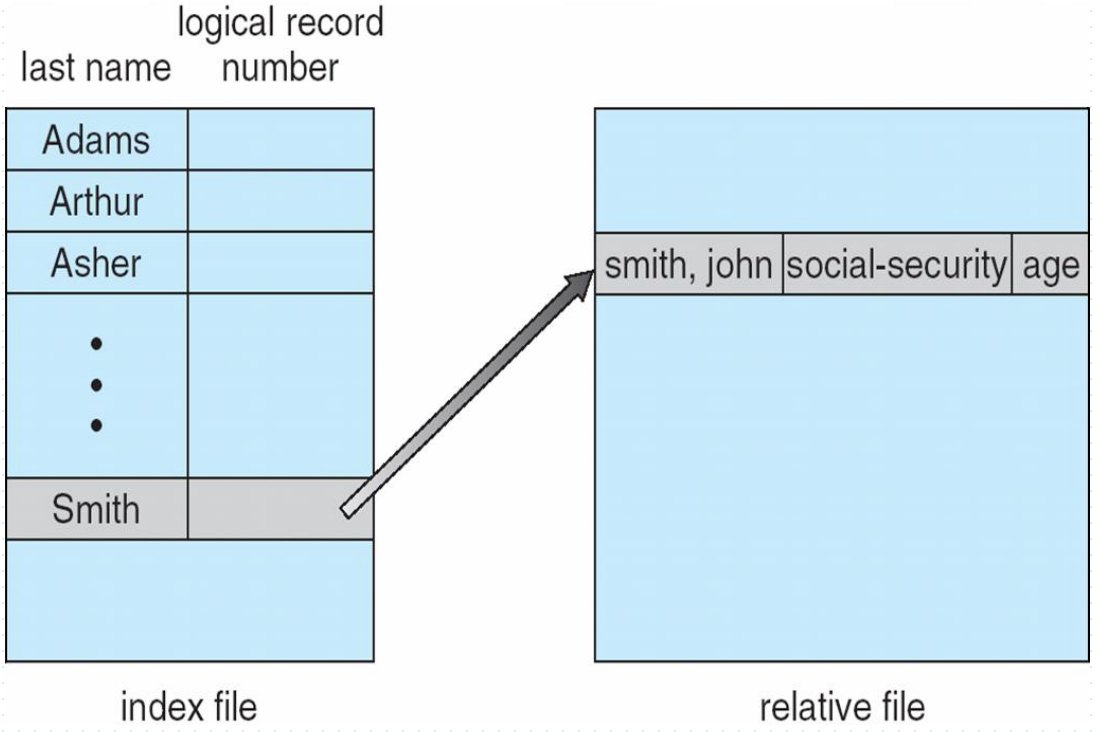
Direct Access (Relative/random access)

- Direct access is implemented on the basis of disk-access model which allows random access to any file block
- A file is made up of a numbered sequence of **fixed-sized** (?) logical records or blocks
- Arbitrary records or blocks in the file can be directly accessed, given the number of the records or blocks
- Block number
  - relative block number, an index relative to the beginning of the file
  - e.g. n = relative block number

    read n
    write n
    position to n
    rewrite n
- Relative block numbers allow OS to decide where file should be placed
  - see allocation problem in Ch 12

Other Access Methods: Index-based Access

- Search-key-based, or Content-based access
- General involve creation of an index for the file, e.g. B+-tree
- Keep index in memory for fast determination of location of data to be operated on (consider UPC code plus record of data about that item)
- If too large, muti-level index: index (in memory) of the index (on disk)
  - e.g. B^+-tree
- Example of **index-based access**: **Index-seek**
  - select ID, name, department
    from instructor
    where lastname='Smith'



System Calls for File Access in Linux

- Open
- Close
- Write
- Read
- Lseek

open()

- **功能描述**：打开或创建文件，在打开或创建文件时可以指定文件的属性及用户的权限等各种参数
- **所需头文件**：#include <sys/types.h>, #include <sys/stat.h>, #include <fcntl.h>
- **函数原型**：int open(const char *pathname, int flags, int perms)
- **参数**
  - pathname：被打开的文件名（可包括路径名，如"dev/ttyS0"）
  - flags(部分）：文件打开方式
    - O_RDONLY：以只读方式打开文件；O_WRONLY：以只写方式打开文件；
    - O_RDWR：以读写方式打开文件；
    - O_APPEND：以添加方式打开文件，在打开文件的同时，文件指针指向文件的末尾
    - O_CREAT：如果打开的文件不存在，创建一个新的文件，并用第三个参数为其设置权限；

    在open()函数中，falgs参数可以通过“|”组合构成；
  - perms：被打开文件的存取权限
    - e.g. S_I(R/W/X)(USR/GRP/OTH)，其中R/W/X表示读写执行权限
- **返回值**：
  - 成功：返回打开/创建文件的文件描述符fd (file descriptor)；失败：返回-1

E.g.
src_file=open(SRC_FILE_NAME,
O_RDWR|O_CREAT,
S_IRUSR|S_IWUSR|S_IRGRP|S_IROTH)

close()

- **功能描述**：关闭一个被打开的文件
- **所需头文件**： #include <unistd.h>
- **函数原型**：int close(int fd)
- **参数**：文件描述符fd
- **返回值**：
  - 成功：0；失败：-1

read()

- **功能描述**：从文件读取数据
- **所需头文件**： #include <unistd.h>
- **函数原型**：ssize_t read(int fd, void *buf, size_t count)
- **参数**：
  - fd：读取数据的文件的文件描述符
  - buf：内存缓冲区，用于存放从文件中读取的数据
  - count：执行一次read操作，**应**读取的字节数
- **返回值**：
  - 返回所读取的字节数；
  - 0，读到文件尾部EOF；-1，出错
- 读操作从指针cfo (current file offset)处开始，在成功返回之前，cfo增加，增量为实际读取到的字节数

E.g.
real_read_len
=read(src_file, src_buff, sizeof(src_buff))

write()

- **功能描述**：向文件写入数据
- **所需头文件**： #include <unistd.h>
- **函数原型**： ssize_t write(int fd, void *buf, size_t count)
- **参数**：
  - fd：写入数据的文件的文件描述符
  - buf：内存缓冲区，存放写入文件的数据
  - count：执行一次write操作，**应**写入文件的字节数
- **返回值**：
  - 成功时，写入文件的字节数
  - 失败时，-1，当磁盘空间满，或者写入数据后超过文件大小限制

E.g.
write(src_file, str, sizeof(str))

- 对于普通文件，写操作始于文件读写指针cfo
- 如果打开文件时使用了参数O_APPEND（从尾部追加写入），则每次写操作都将数据写入文件末尾。成功写入后，cfo 增加，增量为实际写入的字节数

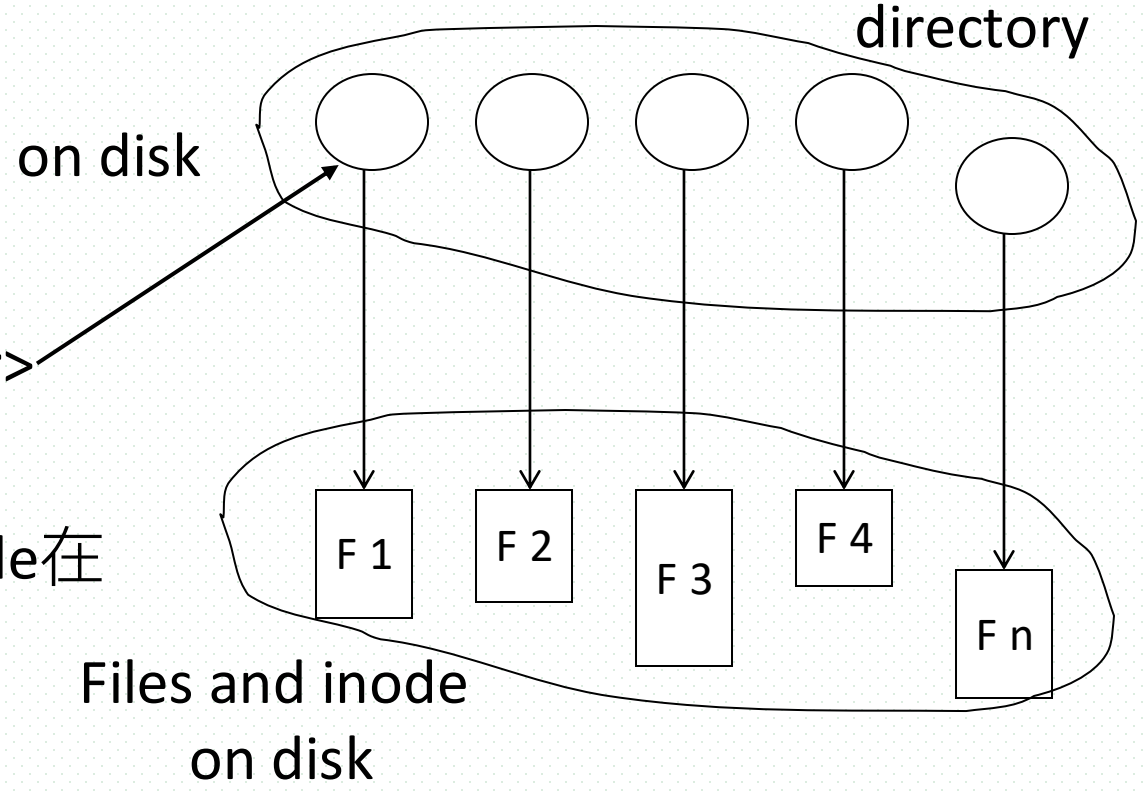
lseek()

- **功能描述**：在文件描述符fd指定的文件中，将文件读写指针cfo定位到相应位置
- **所需头文件**： #include <unistd.h>, #include <sys/types.h>
- **函数原型**：off_t lseek(int fd, off_t offset, int whence)
- **参数**：
  - fd：文件描述符
  - offset：偏移量，读写操作所需要移动的距离，单位是字节byte
    - 值为正，指针移向文件尾部；值为负，指针移向文件头部
  - whence：定位参照点
    - SEEK_SET：参照点为文件开始处0，定位后cfo新位置为偏移量offset，即cfo=offset
    - SEEK_CUR：参照点为读写指针cfo的当前位置，定位后cfo新位置为其当前位置加上偏移量offset，即cfo=cfo+offset
    - SEEK_END：参照点为文件结尾，新位置为文件大小加上偏移量offset，即cfo=size-of(fd)+offset
- **返回值**：
  - 成功，返回cfo的新位置，即返回当前读取位置在文件中的绝对位置；
  - 失败，-1

E.g.
lseek(src_file, OFFSET, SEEK_SET)

10.3 Directory and Disk Structure

- Directories are used to organize a large number of files in systems, which consists of a collection of directory entries containing information about all files
  - set of directory entries (e.g. dentry in Linux), pointing to file-control-block (FCB) on disk
    entry : file-name → file description information (FCB)
- The directory can be viewed as a symbol table that translating file names into their directory entries
- Both the directory structure and the files reside on disk
- In Linux,
  - directory entry : <file_name, inode_number>
  - inode_number → inode storing FCB on disk
    - 根据inode号，定位文件的索引节点inode在磁盘上的位置

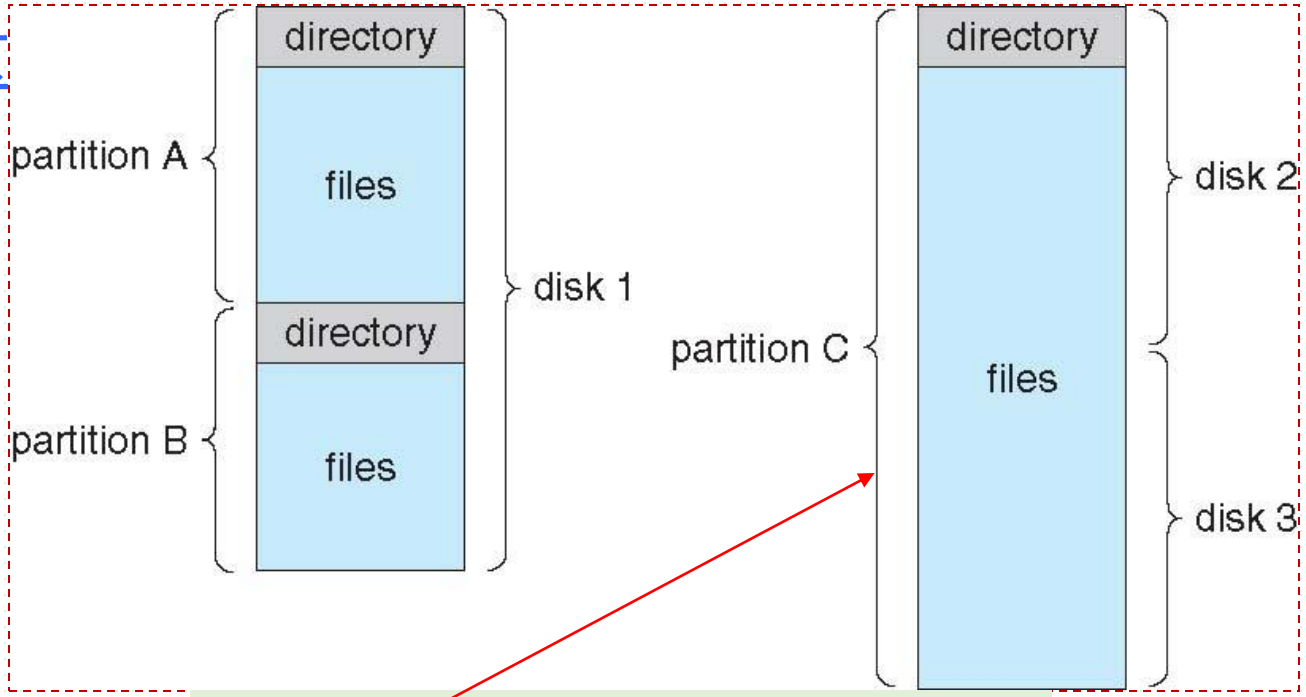


inode in Linux

- inode
  - index node, 索引节点
  - storing file description information on disk, i.e. file control block FCB

Disk (Logical) Structure

- Disk can be subdivided into **partitions/分区**
  - partitions also known as minidisks, slices
- Several disk can be combined to a partition
- Disk or partition can be
  - **raw,** used without a file system, or
    - e.g. create database
  - **formatted** with a file system
- A partition containing file system is known as a **volume/卷**
- Each volume containing file system also tracks that file system's info in **device directory** or **volume table of contents/卷目录表**
- Disks or partitions can be RAID protected against failure



volume = partition + file system

Operations Performed on Directory

- search for a file
- create a file
- delete a file
- list a directory
- rename a file
- traverse/遍历 the file system

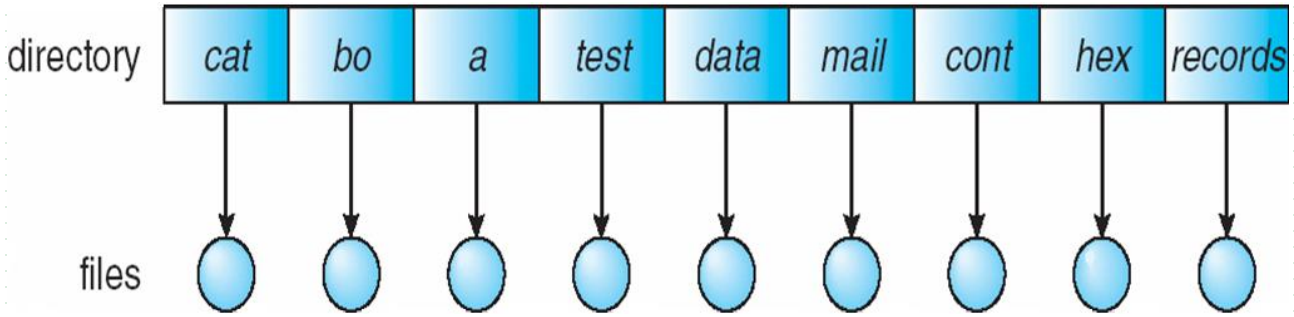
Directory Organization

Organize the directory (logically) to obtain

- Efficiency – locating a file quickly
- Naming – convenient to users
  - two users can have same name for different files
  - the same file can have several different names
- Grouping – logical grouping of files by properties, (e.g., all Java programs, all games, …)

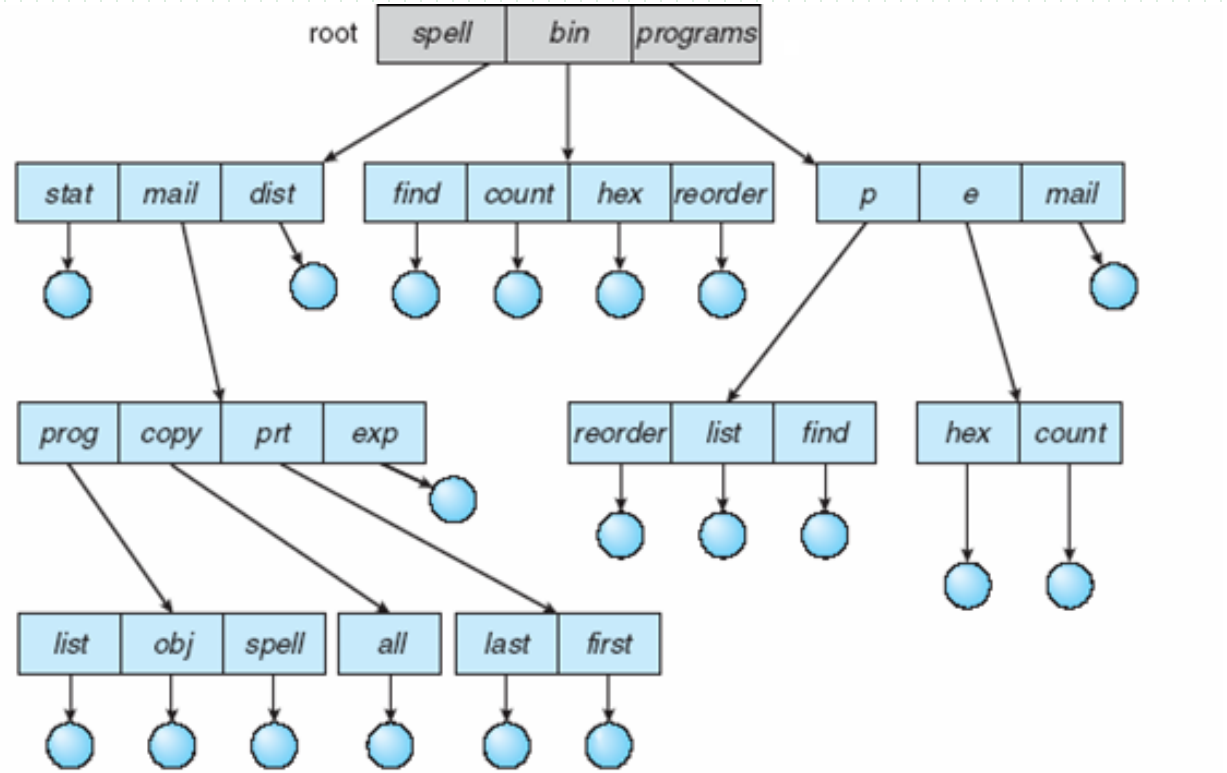
Directory Organization

- Logical structures of the directory
  - single-level directory
  - two-level directory
  - tree-structure directory
  - acyclic-graph directories
  - general graph directories

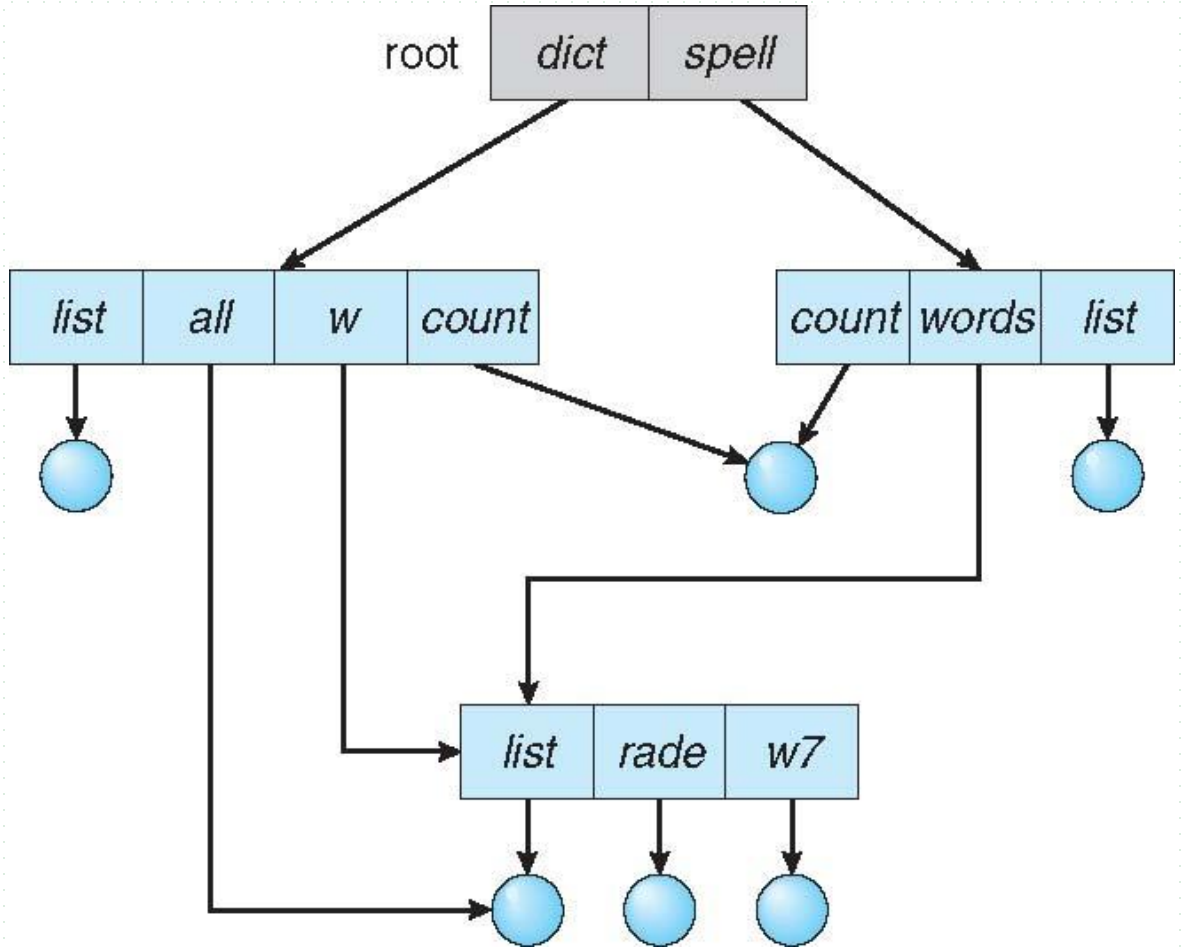


single-level directory

Directory Organization



Tree-Structured Directories



Acyclic-Graph Directories

Linux File System

Link in Unix/Linux【略】

- Link/链接
  - a way, exemplified by Unix/Linux, for sharing files and subdirectories among processes
  - a link is effectively a pointer to another file or subdirectory
- For example, a link may be implemented as an absolute or a relative path name. When a reference to a file is made, we search the directory. If the directory entry is marked as a link, then the name of the real file is included in the link information. We resolve the link by using that path name to locate the real file. Links are easily identified by their format in the directory entry (or by having a special type on systems that support types) and are effectively indirect pointers.
- The operating system ignores these links when traversing directory trees to preserve the acyclic structure of the system

Linux文件中的链接link

- 链接
  - 指向文件或目录的指针
  - 目的：为系统中已有文件指定的另外一个可用于访问该文件的名称/访问路径，以支持多个用户共享访问同一文件
  - 对这个新文件名/链接，可以指定不同的访问权限
- 当链接指向文件目录时，用户可以利用该链接直接进入被链接的目录，快速访问文件，无需输入繁琐冗长的文件路径名
- 删除链接时，不会破坏原有文件目录和文件
- 包括硬链接、软链接

硬链接

- 硬链接
  - 只能引用同一文件系统中的文件，e.g. 多个用户共享访问同一个ext4文件
  - 硬链接引用/访问的是被共享的文件在文件系统中的物理索引inode
  - 移动或删除原始文件时，硬链接不会被破坏
  - 用户根据硬链接访问共享文件时，无需访问原始文件的权限，也不会显示原始文件的位置，这样有助于文件的安全
- 被共享文件的inode中有引用计数器link count，每当该文件被一个用户引用/访问并建立硬链接，link count值加1，count值代表了共享访问该文件的用户数；当用户close共享文件时，count值减1
- 当删除一个文件时，如果该文件有相应的硬链接——即该文件被多个用户共享，该文件依然会保留，直到所有对它的引用都被删除，即count=0，没有用户还需要访问该文件

软链接/符号链接

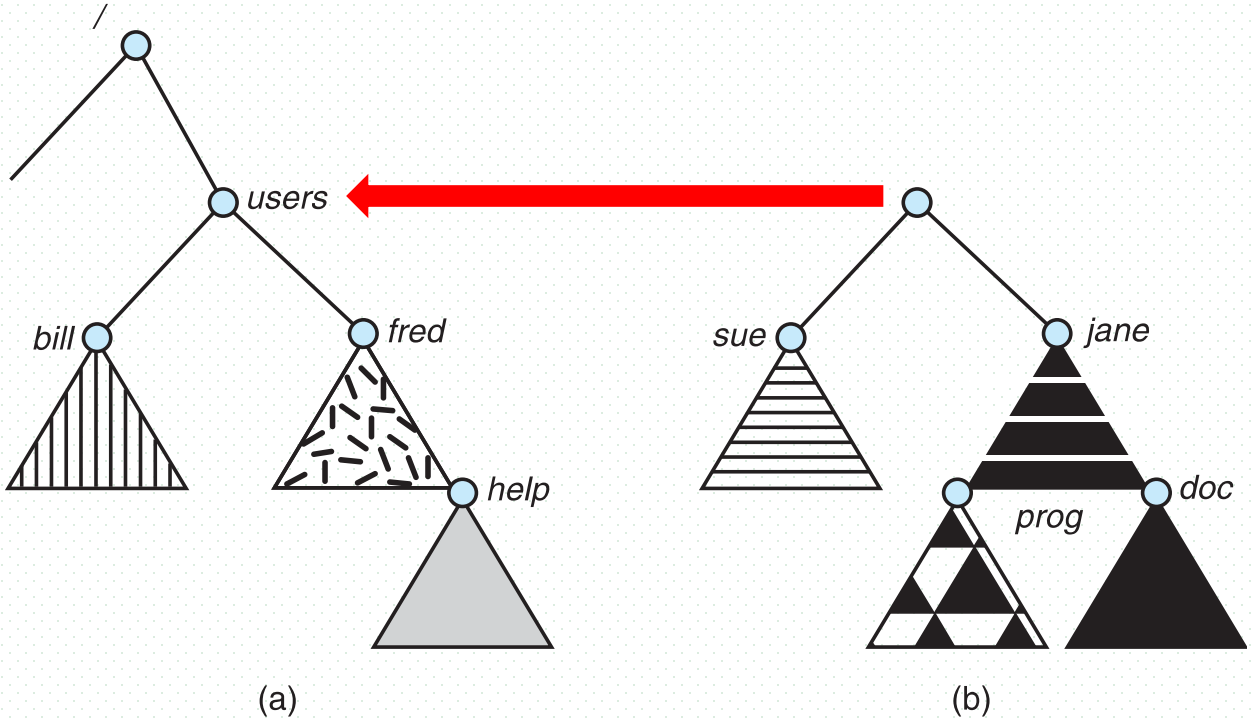
- 软链接
  - 新建立一个文件，包括该文件的inode，专门用于指向被共享文件
  - 类似于windows 下的“快捷方式”
- 删除软连接文件，不会影响被共享的实体原文件；删除实体原文件，则相应的软连接不可用，e.g. 此时cat软链接文件，系统提示“没有该文件或目录”

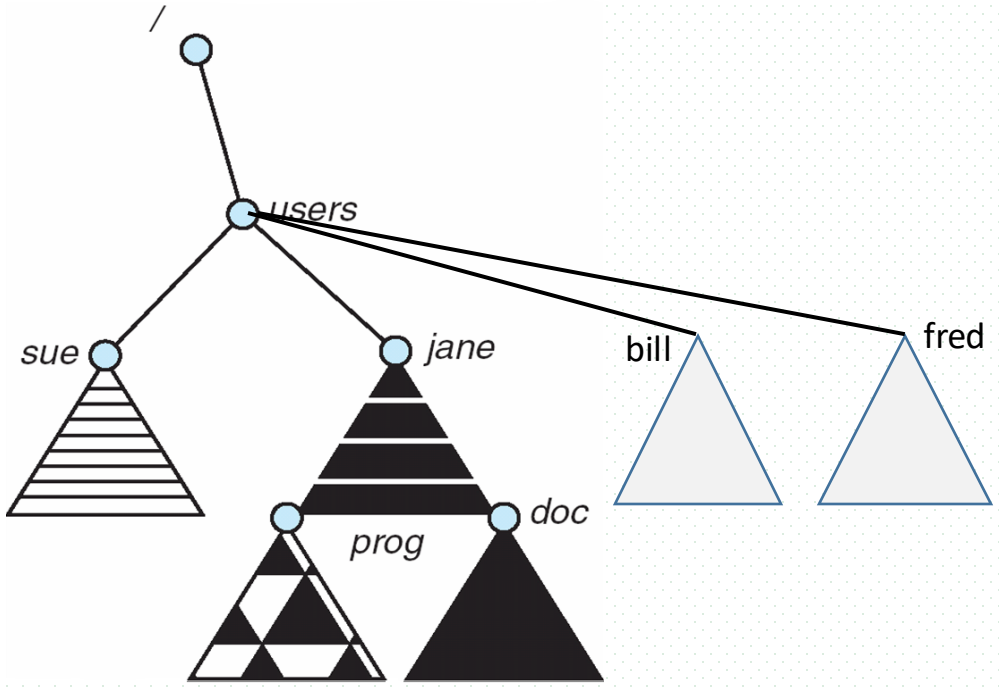
- 创建硬链接不会建立新的inode，而是将被共享的源文件的inode link count增加1，硬链接是不可以跨越文件系统的
  - 硬链接实际上是为被共享文件建一个别名，链接文件和原文件实际上是同一个文件
  - 通过ls -i来查看链接文件和源文件，发现其inode号是同一个，说明它们是同一个文件
- 创建软链接建立一个新文件及其inode，但其inode结构与被共享源文件的inode不同，只是一个指明被共享源文件的字符串信息/指针，软链接可以跨文件系统
- 软链接可以对一个不存在的文件名(filename)进行链接，硬链接则不可以（其文件必须存在，inode必须存在）；软链接可以对目录进行链接，硬链接不可以
- 两种链接可以通过命令 ln 来创建。ln 默认创建的是硬链接。使用 -s 开关可以创建软链接

10.4 File System Mounting

- OS can contain one or more file systems
- Mounting
  - 挂载
- A file system must be **mounted** before it can be accessed
- A unmounted file system (i.e., Fig. 11-11(b)) is mounted at a **mount point,** e.g. /user

Linux启动时，创建（虚拟）文件系统的根目录/，之后逐步将各种文件系统挂载到根目录/下不同的子目录中；
用户也可以挂载、卸载文件系统

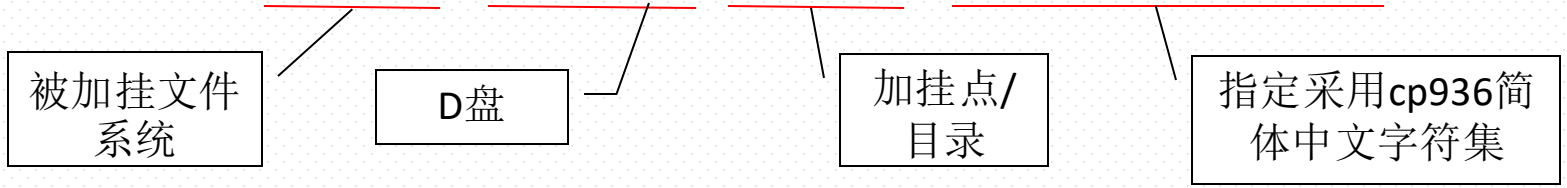




Example

- Mount in Linux
  - 格式：mount [-参数] [设备名称] [挂载点]
  - 用man mount命令可以得到mount命令参数的详细解释
- 机器上同时安装Linux和Windows XP，其中Windows的C盘采用了NTFS文件系统、D盘采用了FAT32文件系统。
- 为了支持在Linux下工作时，对Windows的C盘和D盘的访问，可在Linux下进行加挂
  - 加挂FAT32文件系统

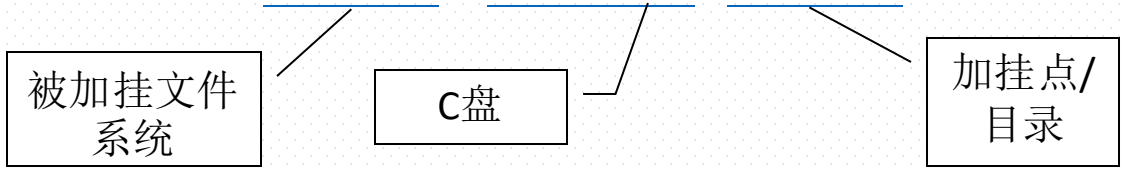
```
mount  -t vfat  /dev/hda6 /mnt/d   -o codepage=936
```



Example

- 加挂NTFS文件系统

```
mount   -t ntfs   /dev/hda2   /mnt/c
```



10.5 File Sharing

- Sharing of files on multi-user systems is desirable
- Sharing may be done through a **protection** scheme
  - owner/user of the shared file/directory, who can change attributes of the shared ones, grant access, and has the most control over the shared ones.
  - group of the shared file/directory, who share the access to the file.
  - which operations can be executed by group members is defined by the owners

File Sharing

- On distributed systems, files may be shared across a network
- Network File System (NFS) is a common distributed file-sharing method
  - distributed database
- If multi-user system
  - **User IDs** identify users, allowing permissions and protections to be per-user
    Group IDs allow users to be in groups, permitting group access rights
  - Owner of a file / directory
  - Group of a file / directory

Remote File Systems

- Uses networking to allow file system access between systems
  - Manually via programs like FTP
  - Automatically, seamlessly using **distributed file systems**
  - Semi automatically via the **world wide web**
- **Client-server** model allows clients to mount remote file systems from servers
  - Server can serve multiple clients
  - Client and user-on-client identification is insecure or complicated
  - **NFS** is standard UNIX client-server file sharing protocol
  - **CIFS** is standard Windows protocol
  - Standard operating system file calls are translated into remote calls
- Distributed Information Systems **(distributed naming services)** such as LDAP, DNS, NIS, Active Directory implement unified access to information needed for remote computing

Protection

- File owner/creator should be able to control
  - what can be done
  - by whom
- Types of access
  - **Read**
  - **Write**
  - **Execute**
  - **Append**
  - **Delete**
  - **List**

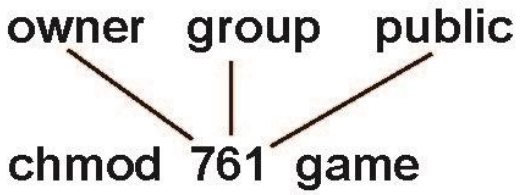
Access Lists and Groups

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

			RWX
a) **owner access**	7	⇒	1 1 1
			RWX
b) **group access**	6	⇒	1 1 0
			RWX
c) **public access**	1	⇒	0 0 1

bits of access list: 3×3

- E.g. Ask manager to create a group (unique name), say G, and add some users to the group.
- For a particular file (say *game*) or subdirectory, define an appropriate access.



Attach a group to a file

```
chgrp      G      game
```

Exercise

- 1 文件F仅有一 个进程打开，当该进程关闭F时，必须的操作是
  - A.删除目录项
  - B.删除内存的文件目录
  - C.删除外存的文件目录
  - D.文件磁盘索引节点链接计数器减1

Exercise

- 2 若文件f1的硬链接为f2，两个进程分别打开f1和f2，获得对应的文件描述符为fd1和fd2，则下列叙述中，正确的是

  I. f1和f2的读写指针位置保持相同

  II.f1和f2共享同一个内存索引结点

  III.fd1和fd2分别指向各自的用户打开文件表中的一项

  - A. 仅III　　B.仅II, III　　C.仅I, II　　D. I, II, III

Exercise

- 3 某文件系统的目录由文件名和索引节点号构成。若每个目录项长度为 64 字节，其中4个字节存放索引节点号，60 个字节存放文件名。文件名由小写英文字母构成，则该文件系统能创建的文件数量的上限为
  - A.2^{26} B.2^{32} C. 2^{60} D.2^{64}

Exercise

- 4 若目录 dir 下有文件 file1，为删除该文件内核不必完成的工作是
  - A.删除 file1的快捷方式
  - B.释放file1的文件控制块
  - C.释放file1占用的磁盘空间
  - D.删除目录 dir 中与 filel 对应的目录项

Exercise

- 5 某文件系统中，针对每个文件，用户类别分为4类：安全管理员、文件主、文件主的伙伴、其他用户；访问权限分为5种：完全控制、执行、修改、读取、写入。若文件控制块中用二进制位串表示文件权限，为表示不同类别用户对一个文件的访问权限，则描述文件权限的位数至少应为
  - A. 5　　B. 9　　C. 12　　D.20

Exercise

- 6 若多个进程共享同一个文件 F，则下列叙述中正确的是
  - A.各进程只能用“读”方式打开文件 F
  - B. 在系统打开文件表中仅有一个表项包含 F 的属性
  - C. 各进程的用户打开文件表中关于 F 的表项内容相同
  - D. 进程关闭 F 时系统删除 F 在系统打开文件表中的表项

4类用户，6种权限

Windows 10 Access-Control List Management

-rw-rw-r--	1 pbg	staff	31200	Sep 3 08:30	intro.ps
drwx------	5 pbg	staff	512	Jul 8 09.33	private/
drwxrwxr-x	2 pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2 pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1 pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1 pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4 pbg	faculty	512	Jul 31 10:31	lib/
drwx------	3 pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3 pbg	staff	512	Jul 8 09:35	test/

A Sample UNIX Directory Listing

Thanks for your attention

北京郵電大學